

Agentic Transition

How AI scaled my output over the last 12 months

I am a solo founder-engineer. Without AI-assisted development, I would not have shipped a production platform with this many services, workers, and data paths in ~18 months. This report shows the scale and how leverage changed over time.

Product: <https://www.plantfolio.garden/> · Architecture: <https://www.plantfolio.garden/support/architecture>

Evidence at a glance

1,635

Git commits (solo founder)

Nov 2023 → Nov 2025

876

TypeScript / TSX files

~65k lines in monorepo

19

Shared packages

Bounded domains for humans + agents

6

Deployable apps & workers

3 Next.js · 2 Railway · Flutter mobile

12+

Production integrations

Auth, pay, search, AI, email, blob, events...

2

PostgreSQL databases

Main + plants (pgvector)

Executive summary

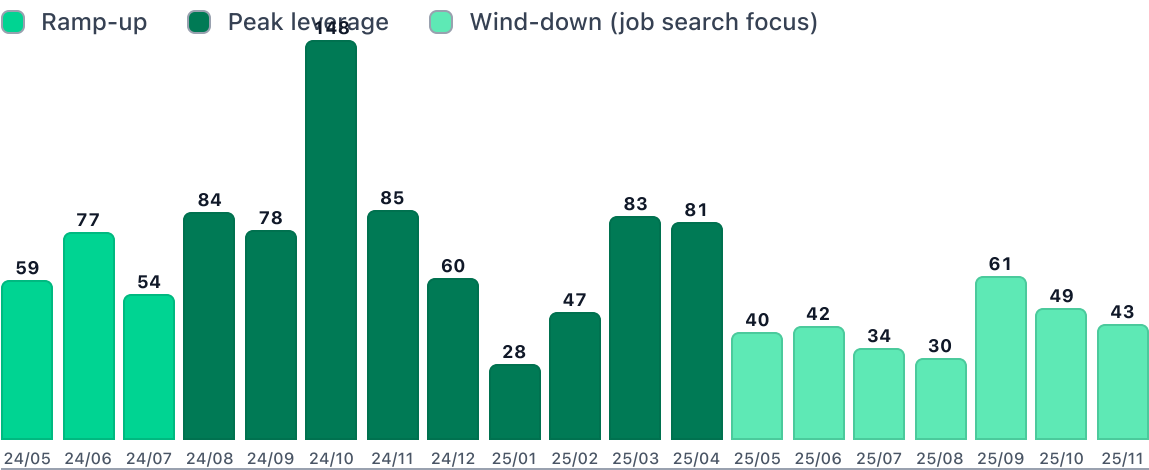
Plantfolio is a Turborepo monorepo I built as founder and sole engineer: three Next.js apps, Flutter mobile, two Railway workers, dual PostgreSQL databases (main + plants with pgvector), and a dozen production integrations, documented at plantfolio.garden/support/architecture.

Cursor (top 1% user) and Claude-class agents are how I hold architecture, migrations, and integration work in my head while shipping. The monorepo is agent-friendly: bounded packages, server actions, typed env, CI on every push.

Commit activity peaked in **2024-10** (148 commits). I slowed active development in late 2025 to focus on hiring; the product and docs remain the proof. **1183** commits in the chart window (May 2024 – Nov 2025).

Output over time

Git commit velocity on Plantfolio (solo author). Values shown on every bar.



Monthly commits on Plantfolio (solo author). Peak month: Oct 2024 (148). Active development intentionally slowed after Nov 2025.

Three phases

● **NOV 2023 – SPRING 2024**

0→1 under agent-assisted velocity

Bootstrapped Plantfolio from customer discovery into a Turborepo monorepo. MongoDB in production first, then a late Postgres migration—scope a solo founder historically would not attempt without AI pair-programming on architecture, migrations, and integration glue.

● **SUMMER 2024 – SPRING 2025**

Peak leverage — platform, not feature

Shipped event-driven workers (change-stream → Inngest), signage render pipeline, dual-database Prisma models, AI identification + RAG, and three Next.js surfaces. Commit velocity peaked Oct 2024 (148 commits). Cursor became the default builder—not a cheat sheet.

● **LATE 2025 – PRESENT**

Intentional pause, proof retained

Stepped back from active Plantfolio development to focus on the job search. The leverage story is the system already in production—not recent commit count. Architecture docs and live product remain the evidence.

Leverage before vs. after agents

Dimension	Pre-agentic solo IC	Last ~12 months
System breadth	One primary app + API; defer workers and mobile	6 deployables, 19 packages, dual DB, async workers
Integrations per year	Handful; long integration spikes	12+ vendors wired with typed env schemas & CI
Data modeling	Would pick one store and live with limits	Mongo → production → migrate to Postgres + pgvector catalog
AI in product	Prototype-only without shipping discipline	Vision ID, Sage assistant, embeddings, RAG in production
How I build	Hand-write boilerplate; context-switch heavy	Cursor top 1%; agents across monorepo boundaries daily
Documentation	Often skipped under time pressure	Public architecture site agents and hires can read

What this scale means

A solo founder historically ships one app and one database. Plantfolio is a multi-surface platform with async workers and AI in the product path.

DEPLOYABLE SURFACES

web

Marketing & public pages

Next.js 16 · Vercel

dashboard

Product UI + Inngest host

Next.js 16 · Vercel

admin

Internal operations

Next.js 16 · Vercel

plantfolio_mobile

Gardeners on mobile

Flutter

change-stream-worker

Postgres LISTEN → Inngest

Node · Railway

render-worker

Sign PDF/ZIP rendering

Express · Railway

SHARED BACKEND PACKAGES

@plantfolio/server

@plantfolio/db

@plantfolio/auth

@plantfolio/ai

@plantfolio/events

@plantfolio/search

@plantfolio/payments

@plantfolio/identification

@plantfolio/media

@plantfolio/email

@plantfolio/plants

@plantfolio/kv

EXTERNAL SERVICES (PRODUCTION)

Clerk

Stripe

Neon (Postgres x2)

Inngest

Algolia

Vercel Blob

Resend

OpenAI / Gemini

Plant.id

Sentry

AWS S3

pgvector

Full diagrams and flows: plantfolio.garden/support/architecture

How I scale myself (workflow)

- Monorepo boundaries (@plantfolio/*) so agents edit in scoped packages—not one giant file.
- Server Actions over public REST: fewer endpoints, clearer mutation surfaces for humans and LLMs.
- Typed env validation per app; `pnpm env:sync` keeps workers and Next apps aligned.
- Vitest + GitHub Actions on every push; agents get fast feedback loops.
- Architecture docs at plantfolio.garden/support/architecture—onboarding for future me and for tools.

This report was built in profile-tools with Cursor—the same agentic workflow it describes.

Andrew Shenstone · May 20, 2026